STATISTICS WITH R PROGRAMMING - ASSESSMENT SOLUTIONS

Name: Pavithra R
Register Number: 192421369

Course Code: ITA0404
Course Name: STATISTICS WITH R PROGRAMMING

1. STUDENT MARKS ANALYZER

Description: Processes a vector of student marks using a `for()` loop and conditional structures to classify students into Distinction, Pass, or Fail categories.

```
# Sample Input
marks <- c(82, 45, 67, 91, 38, 76, 54, 49, 88, 72)

# Initialize counters
distinction_count <- 0
pass_count <- 0
fail_count <- 0

# Process each mark using a for() loop
for (m in marks) {
  if (m >= 75) {
    distinction_count <- distinction_count + 1
  } else if (m >= 50 && m <= 74) {
    pass_count <- pass_count + 1
  } else {
    fail_count <- fail_count + 1
  }
}

# Display total number of students in each category
cat("--- Student Marks Analysis ---
")
cat("Distinction (>= 75):", distinction_count, "
")
cat("Pass (50 - 74):", pass_count, "
")
cat("Fail (< 50):", fail_count, "
")
```

2. ATM WITHDRAWAL CHALLENGE

> **Description:** Simulates continuous ATM transactions with balance checks, denomination validation (multiples of ₹100), exit conditions, and flow control using `while()`, `break`, and `next`.

```
# Initial balance
balance <- 10000

cat("--- Welcome to ATM Simulation ---
")
cat("Initial Balance: ₹", balance, "

")

# Use while() loop to simulate continuous transactions
while (TRUE) {
  # Taking input from user (using readline for interactive execution)
  withdrawal <- as.numeric(readline(prompt = "Enter withdrawal amount (Enter 0 to exit): "))

  # Stop when the user enters 0
  if (withdrawal == 0) {
    cat("Transaction exited. Thank you!
")
    break
  }

  # Check for insufficient balance
  if (withdrawal > balance) {
    cat("Insufficient Balance

")
    next
  }

  # Check if amount is a multiple of 100
  if (withdrawal %% 100 != 0) {
    cat("Invalid Amount (Must be a multiple of ₹100)

")
    next
  }

  # Deduct and display remaining balance
  balance <- balance - withdrawal
  cat("Withdrawal Successful! Remaining Balance: ₹", balance, "

")
}
```

3. MATRIX DIAGONAL PROCESSING

> **Description:** Traverses a square matrix using nested `for()` loops to extract main and secondary diagonal elements, calculates their sums, and compares them.

```
# Sample Input Matrix
m <- matrix(c(5, 2, 8,
              1, 7, 4,
              6, 3, 9), nrow = 3, byrow = TRUE)

n <- nrow(m)
main_diag_sum <- 0
sec_diag_sum <- 0
main_diag_elements <- c()
sec_diag_elements <- c()

# Nested for() loops to traverse the matrix
for (i in 1:n) {
  for (j in 1:n) {
    # Main diagonal condition
    if (i == j) {
      main_diag_sum <- main_diag_sum + m[i, j]
      main_diag_elements <- c(main_diag_elements, m[i, j])
    }
    # Secondary diagonal condition
    if (i + j == n + 1) {
      sec_diag_sum <- sec_diag_sum + m[i, j]
      sec_diag_elements <- c(sec_diag_elements, m[i, j])
    }
  }
}

# Display results
cat("--- Matrix Diagonal Processing ---
")
cat("Main Diagonal Elements:", paste(main_diag_elements, collapse = ", "), "
")
cat("Sum of Main Diagonal:", main_diag_sum, "

")

cat("Secondary Diagonal Elements:", paste(sec_diag_elements, collapse = ", "), "
")
cat("Sum of Secondary Diagonal:", sec_diag_sum, "

")

# Determine which diagonal has the greater sum
if (main_diag_sum > sec_diag_sum) {
  cat("Conclusion: Main diagonal has the greater sum.
")
} else if (sec_diag_sum > main_diag_sum) {
  cat("Conclusion: Secondary diagonal has the greater sum.
")
} else {
  cat("Conclusion: Both diagonals have equal sums.
```

```
")
}
```

4. LOOPING OVER A LIST OF EMPLOYEE RECORDS

> **Description:** Iterates through a heterogeneous list of employee records, handles incomplete entries using `next`, filters high earners (> ₹50,000), aggregates departmental statistics, and identifies the top earner.

```
# Sample Structure (including one incomplete record to test 'next')
employees <- list(
  list(name="Arun", dept="CSE", salary=55000),
  list(name="Bala", dept="ECE", salary=48000),
  list(name="Chitra", dept="CSE", salary=62000),
  list(name=NULL, dept=NULL, salary=NULL), # Incomplete record
  list(name="Deepa", dept="IT", salary=51000),
  list(name="Ezhil", dept="ECE", salary=45000)
)

cse_count <- 0
ece_count <- 0
it_count <- 0

highest_salary <- 0
highest_paid_employee <- ""

cat("--- Employees with Salary > ₹50,000 ---
")

# Process each employee using a for() loop
for (emp in employees) {
  # Use next to skip incomplete records
  if (is.null(emp$name) || is.null(emp$dept) || is.null(emp$salary)) {
    next
  }

  # Display employees with salary > 50000
  if (emp$salary > 50000) {
    cat("Name:", emp$name, "| Dept:", emp$dept, "| Salary: ₹", emp$salary, "
")
  }

  # Count employees in each department
  if (emp$dept == "CSE") {
    cse_count <- cse_count + 1
  } else if (emp$dept == "ECE") {
    ece_count <- ece_count + 1
  } else if (emp$dept == "IT") {
    it_count <- it_count + 1
  }

  # Find highest salary
  if (emp$salary > highest_salary) {
    highest_salary <- emp$salary
    highest_paid_employee <- emp$name
  }
}

cat("
--- Department Statistics ---
```

```
")
cat("CSE Department Count:", cse_count, "
")
cat("ECE Department Count:", ece_count, "
")
cat("IT Department Count:", it_count, "
")

cat("
--- Top Earner ---
")
cat("Highest Paid Employee:", highest_paid_employee, "with Salary ₹", highest_salary, "
")
```

5. CUSTOM CHALLENGE: E-COMMERCE INVENTORY & PRICE MATRIX VALIDATOR

> **Problem Statement:** Write an R program that processes an inventory grid (matrix) representing product prices across 3 stores and 3 regions. Simultaneously, iterate through a list of product structures using a `for()` loop, skip discontinued items using `next`, and use a nested loop to flag products whose prices exceed ₹500.

```
# 1. Vector and Matrix setup
store_names <- c("Store_A", "Store_B", "Store_C")

# Matrix of prices: rows = stores, columns = product categories
price_matrix <- matrix(c(450, 520, 610,
                         300, 480, 750,
                         600, 510, 490), nrow = 3, byrow = TRUE)

# 2. List of product inventory records
inventory_list <- list(
  list(item = "Laptop", price = 55000, status = "Active"),
  list(item = "Mouse", status = "Discontinued"), # Incomplete/Discontinued item
  list(item = "Keyboard", price = 1200, status = "Active"),
  list(item = "Monitor", price = 8500, status = "Active")
)

cat("--- Custom Challenge Execution ---

")

# Processing List with 'next' and 'for'
cat("Active Inventory Items (> ₹1000):
")
for (prod in inventory_list) {
  if (is.null(prod$price) || prod$status == "Discontinued") {
    next # Skip discontinued items
  }
  if (prod$price > 1000) {
    cat("-", prod$item, ": ₹", prod$price, "
")
  }
}

cat("
--- Matrix Processing & Nested Loops ---
")

# Nested loops to process matrix and flag expensive items (> 500)
for (i in 1:nrow(price_matrix)) {
  store_total <- 0
  cat("Checking", store_names[i], "prices:
")
  for (j in 1:ncol(price_matrix)) {
    val <- price_matrix[i, j]
    store_total <- store_total + val
    if (val > 500) {
      cat("  * Product Category", j, "costs ₹", val, "(High Price Flag)
")
```

```
    }
  }
  cat("Total Revenue/Value for", store_names[i], ": ₹", store_total, "

")
}
```